Day 174 — Line-by-Line Syntax Explanation

MD Mutasim Billah | 52-Week Data Science to ML/AI Roadmap | Week 25, Day 7 reference

Watch: Reproducible machine learning and science with python (Prabhant Singh, PyData) | Using Reproducible Experiments To Create Better Machine Learning Models (PyData Global 2021)

`day174_reproducible_study.py`, segment by segment, in the order `main()` uses things. Today has no new parts and no exercise file: it imports Days 168 to 173's exercises unchanged and builds one study from them. The script runs itself in child processes (python day174_reproducible_study.py --study NAME). Code boxes are cut from the real script by program (inline comments removed). Purple **MATH** boxes come before code that implements a formula. A glossary is at the end.

MATH BOX LEGEND

Symbols: T = a tracker · D = the data arrays · p = one configuration · u = its point in the unit cube · t = trial number · A, B, C, S1, U1, U2, T1, D30 = studies · O = one study's outcome (settings → record)

Segment 0 — Setup

Imports and constants

CODE

```
import argparse
import contextlib
import hashlib
import io
import json
import os
import platform
import shutil
import subprocess
import sys
import time
import numpy as np
HERE = os.path.dirname(os.path.abspath(__file__))
ROOT = os.path.dirname(HERE)
sys.path.insert(0, HERE)
sys.path.insert(0, os.path.join(ROOT, "week-24-model-evaluation-statistics"))
with contextlib.redirect_stdout(io.StringIO()):
    import day161_exercise as d161
    import day168_exercise as d168
    import day169_exercise as d169
    import day170_exercise as d170
    import day171_exercise as d171
    import day172_exercise as d172
    import day173_exercise as d173
d126 = d171.d126
RUNS = os.path.join(HERE, "day174_runs")
PIN = os.path.join(HERE, "day174_environment.json")
REQ = os.path.join(HERE, "day174_requirements.txt")
CV_ROWS, STOP_ROWS = 3000, 6000
K = 3
N_RANDOM, N_BO = 8, 12
CANDIDATES = 2000
FIND_STEPS = 300
CRASH_AT = 13
LR_BELOW = 300
THREADS = "4"
```

- `import argparse` — reads command-line options such as --study A --seed 1.
- `import subprocess` — starts another program, here a new Python running this same file.

- `import day168_exercise as d168 ... day173_exercise as d173` — the week's six parts, imported unchanged.
- `LR_BELOW = 300` — how far below the finder's suggestion the search reaches; the first draft used 30.

Segment 1 — One study (child process)

load_data, space_from and cv_score

CODE

```
def load_data():
    tr, te = d161.load_adult(); fi, va = d161.split_rows(len(tr["y"]))
    ref = {k: v[fi] for k, v in tr.items()}
    X = d161.onehot_matrix(tr, ref).astype(np.float32); y = tr["y"].astype(np.float32)
    Xt = d161.onehot_matrix(te, ref).astype(np.float32); yt = te["y"].astype(np.float32)
    p = np.random.RandomState(0).permutation(fi)
    return dict(Xc=X[p[:CV_ROWS]], yc=y[p[:CV_ROWS]], Xs=X[p[-STOP_ROWS:]], ys=y[p[-STOP_ROWS:]],
                Xv=X[va], yv=y[va], Xt=Xt, yt=yt)
def space_from(s, below=LR_BELOW):
    return (("lr", "log", s / below, s * 3), ("patience", "int", 1, 20), ("batch", "logint", 32,
        256))
def cv_score(D, p, folds, seed):
    def fit_score(tr, te):
        es = d171.EarlyStopper(p["patience"])
        net, acts, curve = d171.train_es(D["Xc"][tr], D["yc"][tr], D["Xs"], D["ys"], d126.Adam,
            p["lr"], seed, es,
            batch=p["batch"])
        return float(d171.row_losses(net, acts, D["Xc"][te], D["yc"][te]).mean())
    return d172.cross_val(folds, fit_score)
```

- `d161.onehot_matrix(te, ref)` — the test file coded with the fitting rows' categories and scaling, like the rest.
- `def space_from(s, below=LR_BELOW):` — Day 168's space format, built around a number found at run time.
- `def fit_score(tr, te):` — a function defined inside another; it can use D, p and seed from the outer one.
- `d172.cross_val(folds, fit_score)` — Day 172 calls fit_score once per fold and averages.

study: the finder and the trials

MATH

$$EI(x) = (best - \mu(x))\,\Phi(z) + \sigma(x)\,\phi(z), \quad z = (best - \mu(x)) / \sigma(x) \quad \text{(Day 169)}$$

CODE

```
def study(name, seed, crash_at=None, unseeded=False, below=LR_BELOW):
    os.makedirs(RUNS, exist_ok=True)
    T = d173.Tracker(os.path.join(RUNS, name + ".jsonl"))
    D = load_data()
    data = d173.fingerprint(*[D[k] for k in sorted(D)])
    code = d173.file_hash(__file__)
    fp = dict(step="lr_finder", steps=FIND_STEPS)
    found = d173.query(T.runs(), where=lambda r: r["params"] == fp)
    if not found:
        with T.run(fp, seed, data, code) as r:
            lrs, raw, sm = d170.lr_find(d170.mlp_stepper(D["Xc"], D["yc"], d126.Adam, seed),
                d170.lr_range(d170.LO, d170.HI, FIND_STEPS))
            r.summary.update(d170.suggest(lrs, sm), lrs=lrs.tolist(), sm=sm.tolist())
        found = d173.query(T.runs(), where=lambda r: r["params"] == fp)
    space = space_from(found[0]["summary"]["min_over_10"], below)
    folds = d172.shuffled_folds(CV_ROWS, K, None if unseeded else seed)
    U0 = d168.random_unit(N_RANDOM, len(space), np.random.RandomState(seed))
```

```
    for t in range(N_RANDOM + N_BO):
        done = {r["params"]["trial"]: r for r in trials_so_far(T)}
        if t in done:
            continue
        hist = [done[i] for i in range(t)]
        if t < N_RANDOM:
            u = U0[t]
        else:
            X = np.array([r["params"]["u"] for r in hist]); y = np.array([r["summary"]["cv_loss"]
                for r in hist])
            C = np.random.RandomState(seed * 1000 + t).rand(CANDIDATES, len(space))
            u = C[d169.propose(X, y, C)[0]]
        p = dict(step="trial", trial=t, u=[float(x) for x in u], **d168.decode(u[None, :],
            space)[0])
        with T.run(p, seed, data, code) as r:
            if t == crash_at:
                raise RuntimeError(f"simulated crash in trial {t}")
            m, s = cv_score(D, p, folds, seed)
            r.summary.update(cv_loss=m, folds=s.tolist())
```

- `if not found:` — an empty list is false: the finder runs only if the log has no finished finder step.
- `{r["params"]["trial"]: r for r in ...}` — a dictionary comprehension: trial number → its record.
- `if t in done: continue` — a resumed study skips trials the log shows finished.
- `hist = [done[i] for i in range(t)]` — only the trials before t: the fix for the first draft's resume bug.
- `u[None, :]` — one point as a one-row matrix, the shape decode expects.
- `dict(step="trial", ..., **d168.decode(...)[0])` — the record's settings: bookkeeping keys plus the decoded values.

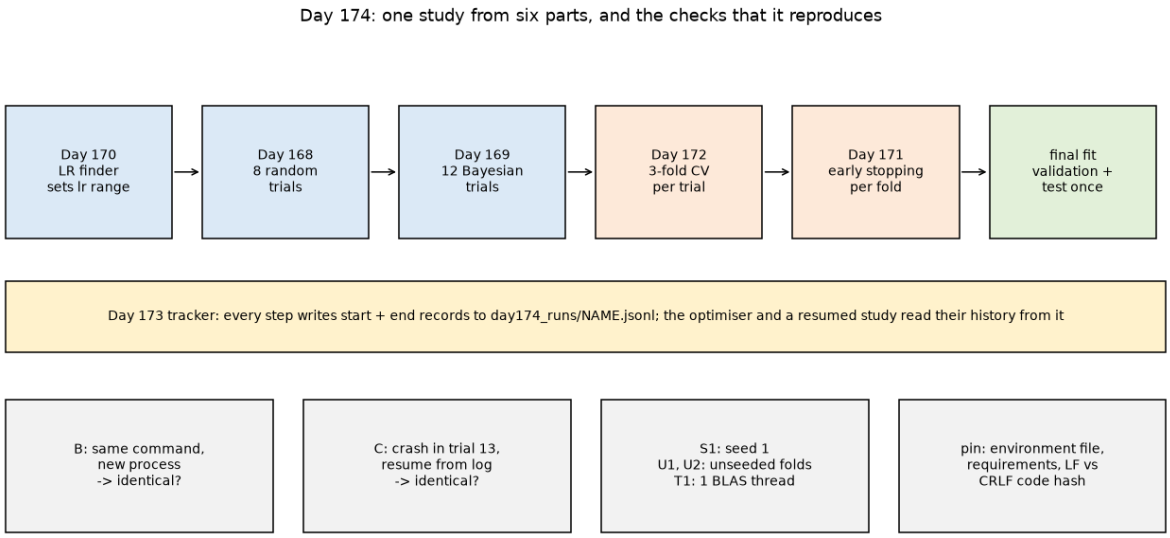
study: the final fit, and child

CODE

```
    best = min(trials_so_far(T), key=lambda r: r["summary"]["cv_loss"])["params"]
    with T.run(dict(best, step="final"), seed, data, code) as r:
        es = d171.EarlyStopper(best["patience"])
        net, acts, curve = d171.train_es(D["Xc"], D["yc"], D["Xs"], D["ys"], d126.Adam,
            best["lr"], seed, es, batch=best["batch"])
        zt = d126.forward(net, D["Xt"], acts)[-1][:, 0]
        r.summary.update(val_loss=float(d171.row_losses(net, acts, D["Xv"], D["yv"]).mean()),
            test_loss=float(d171.row_losses(net, acts, D["Xt"], D["yt"]).mean()),
            test_acc=float(np.mean((zt > 0) == (D["yt"] > 0.5))),
            weights=d173.fingerprint(*[a for W, b in net for a in (W, b)]),
            best_epoch=es.best_epoch + 1)
def child(argv):
    a = argparse.ArgumentParser()
    a.add_argument("--study"); a.add_argument("--seed", type=int, default=0)
    a.add_argument("--crash-at", type=int, default=None); a.add_argument("--unseeded",
        action="store_true")
    a.add_argument("--lr-below", type=float, default=LR_BELOW)
    o = a.parse_args(argv)
    study(o.study, o.seed, o.crash_at, o.unseeded, o.lr_below)
```

- `dict(best, step="final")` — a copy of the best trial's settings with step replaced, so its id differs.
- `(zt > 0) == (D["yt"] > 0.5)` — a logit above 0 means a predicted probability above one half.
- `[a for W, b in net for a in (W, b)]` — every weight matrix and bias, flattened into one list for the fingerprint.
- `a.add_argument("--unseeded", action="store_true")` — a flag: True if present, False if not.
- `type=int, default=None` — the option's text is converted to int; absent means None.

SYSTEM DIAGRAM



The study's pipeline and the checks.

Segment 2 — The experiments (parent process)

launch, outcome, compare and max_gap

CODE

```
def launch(name, *args, threads=THREADS):
    env = dict(os.environ, OPENBLAS_NUM_THREADS=threads, OMP_NUM_THREADS=threads,
        MKL_NUM_THREADS=threads,
               PYTHONHASHSEED="0")
    t0 = time.time()
    p = subprocess.run([sys.executable, __file__, "--study", name, *args], env=env,
        capture_output=True, text=True)
    return p.returncode, time.time() - t0, p.stderr.strip().split("\n")[-1]
def outcome(name):
    T = d173.Tracker(os.path.join(RUNS, name + ".jsonl"))
    R = d173.query(T.runs())
    return {json.dumps(r["params"], sort_keys=True): r for r in R}
def compare(a, b):
    A, B = outcome(a), outcome(b)
    keys = [k for k in A if json.loads(k).get("step") != "lr_finder"]
    same = [k for k in keys if k in B and B[k]["summary"] == A[k]["summary"]]
    return len(same), len(keys), A, B
def max_gap(A, B):
    gaps = [abs(A[k]["summary"][m] - B[k]["summary"][m]) for k in A if k in B
            for m in ("cv_loss", "val_loss", "test_loss") if m in A[k]["summary"]]
    return sum(k in B for k in A), max(gaps) if gaps else None
```

- `dict(os.environ, OPENBLAS_NUM_THREADS=threads, ...)` — a copy of this process's environment with some variables set; the child inherits it.
- `subprocess.run([sys.executable, __file__, ...], env=env, capture_output=True, text=True)` — runs this Python on this file and waits; its output is kept as text.
- `p.stderr.strip().split("\n")[-1]` — the last line of the child's error output: the exception, if it crashed.
- `json.dumps(r["params"], sort_keys=True)` — settings as text, so they can be a dictionary key.
- `B[k]["summary"] == A[k]["summary"]` — dictionaries are equal only if every key and every value is.
- `next(r for ... if ...)` — the first item a generator yields.

demo_breaks: four variants

CODE

```
for name, args, th, what in (("S1", ("--seed", "1"), THREADS, "study seed 1 instead of 0"),
    ("U1", ("--unseeded",), THREADS, "folds with no seed, first run"),
    ("U2", ("--unseeded",), THREADS, "folds with no seed, second run"),
    ("T1", (), "1", "seed 0, 1 BLAS thread instead of 4")):
    code, sec, err = launch(name, *args, threads=th)
    n, m, A, V = compare("A", name)
    f, fa = final(V)["summary"], final(A)["summary"]
    b = json.loads(next(k for k in V if json.loads(k)["step"] == "final"))
    print(f"  {name} ({what}): {n} of {m} of A's steps reproduced; chose lr {b['lr']:.2e},
        patience {b['patience']}, "
          f"batch {b['batch']}; test log loss {f['test_loss']:.4f} (A {fa['test_loss']:.4f}),
              weights {f['weights']}")
    sh, g = max_gap(A, V)
    print(f"      steps with A's exact settings: {sh} of {len(A)}; largest score difference among
        them: "
          f"{'none shared' if g is None else f'{g:.1e}'}")
    out[name] = V
```

- `for name, args, th, what in ((...), (...)):` — each tuple is unpacked into four names.
- `launch(name, *args, threads=th)` — *args spreads the tuple of options into separate arguments.
- `{'none shared' if g is None else f'{g:.1e}'}` — a conditional inside an f-string; :.1e is scientific notation.

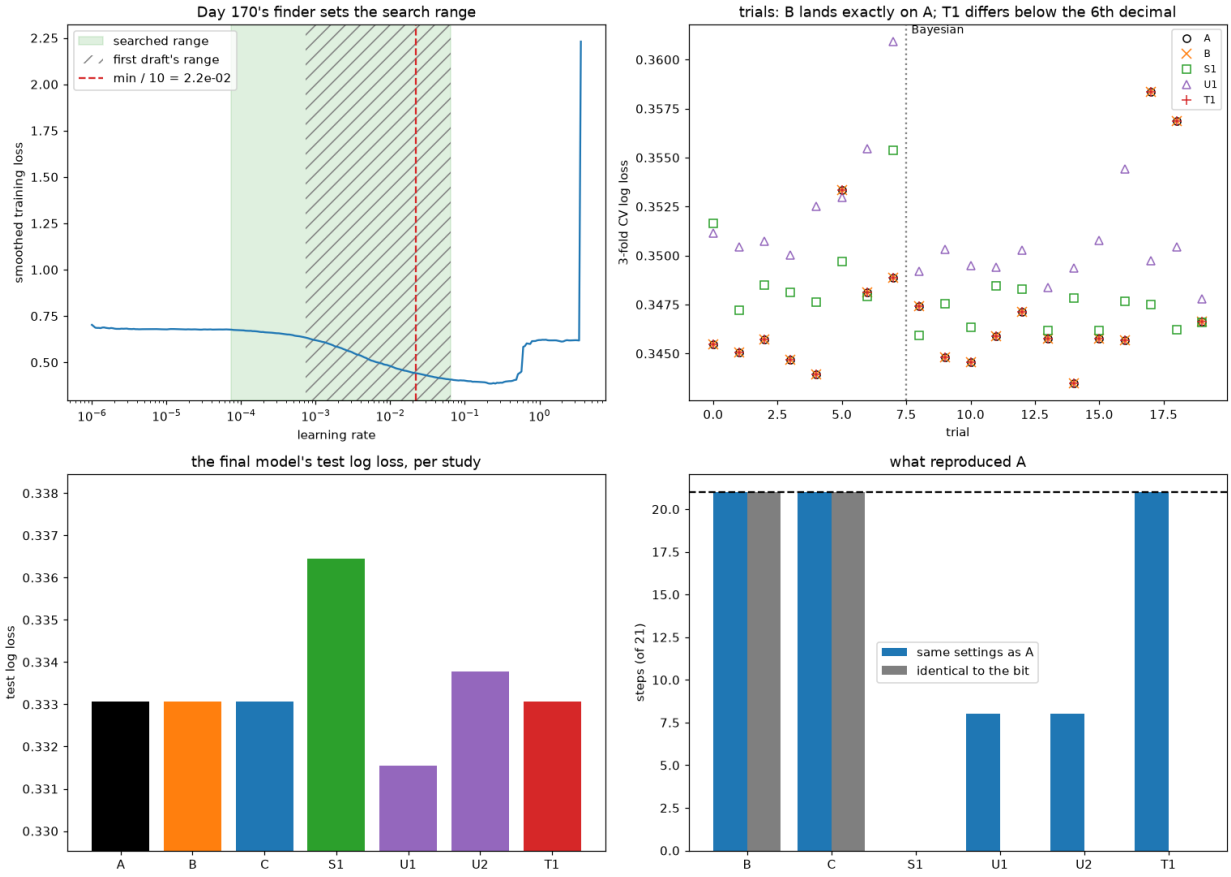
demo_pinning: the pin file and the line-ending trap

CODE

```
env = environment()
if not os.path.exists(PIN):
    json.dump(env, open(PIN, "w", encoding="utf-8"), indent=1, sort_keys=True)
    print(f"  wrote {os.path.basename(PIN)}")
pinned = json.load(open(PIN, encoding="utf-8"))
diff = {k: (pinned.get(k), v) for k, v in env.items() if pinned.get(k) != v}
print(f"  pinned: {', '.join(f'{k} {v}' for k, v in sorted(pinned.items()))}")
print(f"  this machine against the pin: {'no differences' if not diff else diff}")
with open(REQ, "w", encoding="utf-8") as f:
    f.write(f"numpy=={env['numpy']}\nmatplotlib=={env['matplotlib']}\n")
print(f"  {os.path.basename(REQ)}: numpy=={env['numpy']}, matplotlib=={env['matplotlib']} (python
    {env['python']})")
rec = json.loads(open(os.path.join(RUNS, "A.jsonl"), encoding="utf-8").readline())
print(f"  every start record carries its environment too, e.g. study A's first: {rec['env']}")
raw = open(__file__, "rb").read()
lf = raw.replace(bytes([13, 10]), bytes([10])); crlf = lf.replace(bytes([10]), bytes([13, 10]))
h = lambda b: hashlib.sha256(b).hexdigest()[:12]
print(f"  code hash of this script on disk {h(raw)} (the hash every record carries); with LF line
    endings {h(lf)}, "
      f"with CRLF line endings {h(crlf)}: the same code, two hashes")
```

- `json.dump(env, open(PIN, "w", ...), indent=1, sort_keys=True)` — writes the dictionary as readable JSON.
- `{k: (...) for k, v in env.items() if ...}` — only the keys whose value differs from the pin.
- `bytes([13, 10])` — the two bytes carriage return and line feed: a Windows line ending (CRLF).
- `raw.replace(bytes([13, 10]), bytes([10]))` — CRLF to LF; replacing LF with CRLF goes the other way.

REAL GRAPH



Real graph from the script.

Quick Syntax Glossary — new or reinforced constructs

- `argparse.ArgumentParser().add_argument(...)` — command-line options.
- `subprocess.run([...], env=..., capture_output=True)` — run another program and wait for it.
- `dict(os.environ, NAME=value)` — a modified copy of the environment.
- `if __name__ == "__main__":` — true when the file is run, not imported; here it also chooses parent or child.
- `{k: v for ...}` — a dictionary comprehension.
- `*args in a call` — spread a tuple into separate arguments.
- `bytes([13, 10])` — bytes from their numbers.

End of Day 174 syntax reference. For the experiments and what they showed, see the Day 174 complete guide.